P04: Makers Makin' It, Act II
Team Name: Organutans
Roster: Kiran Soemardjo (PM), Mustafa Abdullah (Dev 1), Yu Lu (Dev 2), Eviss Wu (Dev 3)
TARGET SHIP DATE: 2026-04-20

Description
We plan to pull data from the Steam API, and use the data to visualize trends in the games that are popular at the moment. The data will also be filterable/sortable by rating, player count, genre, tag, etc. Additionally, the user will be able to enter their Steam ID, and get recommendations based on the games in their library, as well as an analysis of personal trends (through the Steam API using the user's ID).

Dependencies
Framework: Bootstrap (FrontEnd)
JavaScript: D3Js
Python: Flask, Sqlite3
APIs: Steam Web API (100k calls/day)

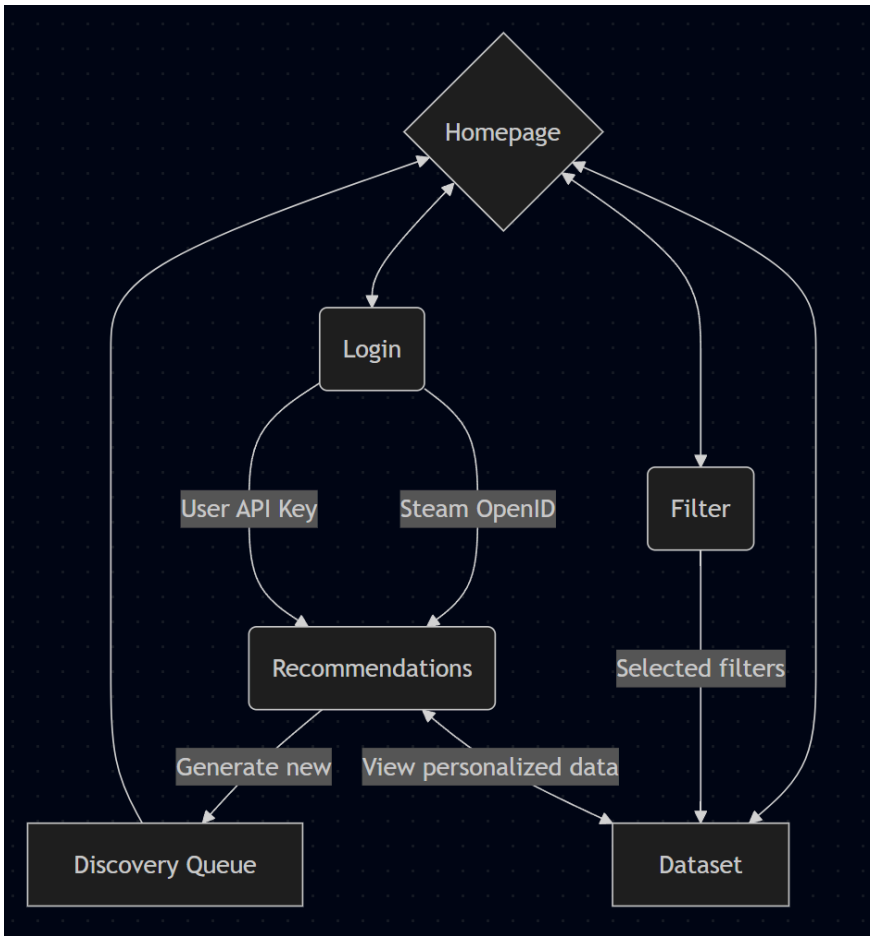
Tasks + Assignments
Project Manager: Kiran Soemardjo
- Flask functionality
- Filtering, sorting, and recommendations

Dev1: Mustafa Abdullah
- Create database structure
- Data processing/parsing

Dev2: Yu Lu
- Implement Javascript for site interactability
- Visualize data with chart.js

Dev3: Eviss Wu
- Frontend Visuals
- UI

Site Map



Routes:

- / *(homepage)*
- /login *(authentication page where Steam ID is inputted – no registration)*
- /recommendations *(user page from authentication)*
- /recommendations/queue *(generate a new discovery queue based on the current user's data)*
- /filter *(manual filter selection)*
- /dataset *(dataset results visualized, reached through* /, /filter,or /recommendations*)*

Front-End Framework: Bootstrap

We chose to use Bootstrap because it has a large amount of pre-built components. Although this may come at the cost of nuance, we agreed upon Bootstrap because these pre-built components offer a way for members that haven't taken the time to learn bootstrap, an easy way and fast way to edit our site's UI. This would be opposed to something like Tailwind, which offers customization in exchange for ease of learning. Also, Bootstrap is relatively human-readable.

Data Visualization Library: chart.js

Due to our data mainly being used to compare games and their monetary success or playercount, we will typically have two axes. This means that a trendline graph, bar graph or pie chart should be enough to suit our needs. We also chose this because we were all familiar with javascript and HTML, which chart.js blends in order to provide its clean-looking chart. Chart.js provides built-in interactivity as well, allowing us to do things like filter our charts for specific times, genres, etc. if need be. On the other hand, chart.js may be slow when it comes to larger datasets and the pre-built charts may look un-unique.

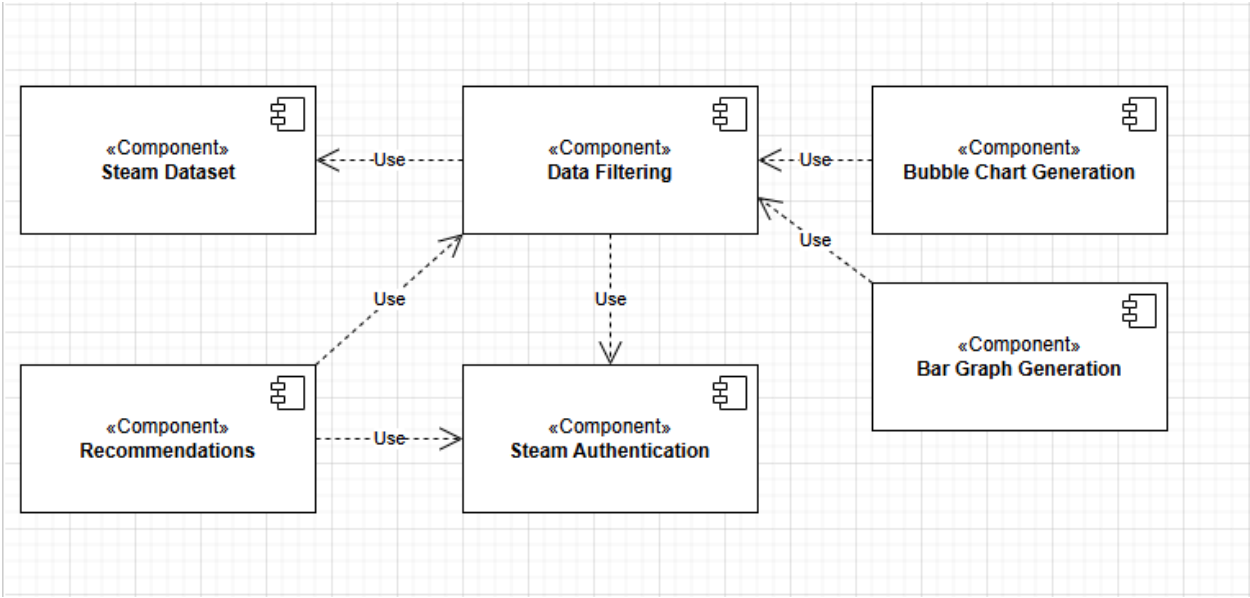
External Resources

https://github.com/MattRyder/SteamAPI

Components

- Steam Dataset
  - Pulled from API and organized with sqlite3
- Data Filtering
  - Handles processing and filtering of data for specific queries
- Bubble Chart Generation
  - Creates bubble graph from filtered dataset for visualizing trends by genre/tag
- Bar Graph Generation
  - Creates bar graph from filtered dataset for visualizing trends over time
- Steam Authentication
  - Authentication of steam ID, pulling user data from the API for use
- Recommendations
  - Uses user library/playtime data to generate recommendations

Component Map



Database Organization

Games		
TEXT	app_id	PK
TEXT	name	
TEXT	release_date	
TEXT	developer	
TEXT	publisher	
BOOLEAN	is_free	
DOUBLE	price	
TEXT	description	
TEXT	genre_list	
TEXT	tag_list	

Player_stats		
INTEGER	app_id	FK games(app_id)
INTEGER	current_players	
INTEGER	peak_players_today	
TEXT	time_feteched	

Users		
TEXT	steam_id	PK
TEXT	api_key	
INTEGER	installed_games	INTEGER DEFAULT 0
DOUBLE	playtime	

Reviews		
INTEGER	app_id	FK REF games(app_id)
INTEGER	review_score	
TEXT	review_score_desc	
INTEGER	total_positive	
INTEGER	total_negative	
INTEGER	total_reviews	